

SkipFold: Test-Time Dynamic Depth for Scalable Biomolecular Inference

Wen Yeong Lee

NVIDIA AI Technology Center (NVAITC) NVIDIA AI Technology Center (NVAITC)

Singapore

lwenyeong21@gmail.com

Darren Tan

Singapore

dhui@nvidia.com

Ian McLoughlin

Singapore Institute of Technology

Singapore

Ian.McLoughlin@singaporetech.edu.sg

Aik Beng Ng

NVIDIA AI Technology Center (NVAITC)

Singapore

aikbengn@nvidia.com

Zhengkui Wang

Singapore Institute of Technology

Singapore

zhengkui.wang@singaporetech.edu.sg

Simon See

NVIDIA AI Technology Center (NVAITC)

Singapore

ssee@nvidia.com

Abstract—Biomolecular foundation models predict protein structure at near-experimental accuracy, but proteome-scale deployment is bottlenecked by the $O(N^3)$ PairFormer, run across four recycling iterations. Hardware-exact kernels (FlashAttention, cuEquivariance) speed each operation but do not reduce the blocks executed. We present SKIPFOLD, a retraining-free test-time dynamic-depth method: Warm-Start Recycling skips the first k PairFormer blocks in later iterations and runs only a short suffix, since the recycled pair representation is already partially refined. On a 50-target CASP16 benchmark (48 over three seeds), the schedule (0, 40, 40, 40) cuts PairFormer block evaluations from 212 to 92 (56%; a $\sim 35\%$ wall-clock reduction over a torch.compile baseline) while holding fidelity to the full four-iteration model high for medium and long proteins (mean TM 0.924, 0.954); short proteins ($N < 300$) show the largest fidelity loss (0.799) and need conservative schedules. Because SkipFold accelerates a fixed pretrained predictor, we measure agreement with the full-model output, not accuracy against experimental structures.

Index Terms—protein structure prediction, test-time compute, dynamic depth, AlphaFold, inference acceleration, PairFormer, recycling

I. INTRODUCTION

The prediction of biomolecular structure from sequence has been transformed by deep learning, with models such as AlphaFold [1], [10] and RoseTTAFold [2], [11] achieving near-experimental accuracy. At the core of these architectures lies the *PairFormer*, a triangular self-attention stack operating over the $N \times N$ pair representation $z \in \mathbb{R}^{N \times N \times c_z}$, where N is the number of residues and c_z the channel dimension. The PairFormer applies 48 sequential blocks, each containing triangle multiplicative updates ($O(N^3 \cdot c)$) and triangle attention ($O(N^2 \cdot c)$ with $c = 128$). This stack is invoked across $T = 4$ recycling iterations, during which the single representation s and pair representation z are iteratively refined. The total computational cost scales as $\sim T \cdot L \cdot O(N^3)$, where $L = 48$ is the block count and T the recycling depth. For a 1,500-residue protein on an NVIDIA H100, a single inference pass consumes over 30 seconds of GPU time in the PairFormer alone.

The dominant approach to accelerating these models has been *hardware-level optimization*: custom CUDA kernels (FlashAttention-4, cuEquivariance) that reorder computation to maximize HBM bandwidth while preserving bit-for-bit numerical fidelity [4], [18]. These techniques yield large speedups: we measure a $2.6\times$ reduction from FA4 alone at $N = 1024$, but they are approaching the physical ceiling: the $O(N^3)$ algorithmic complexity remains unchanged. Further gains require *algorithmic* innovation that reduces the number of operations performed, not merely their scheduling.

We observe that the recycling mechanism in AlphaFold-style architectures creates an opportunity for algorithmic acceleration that does *not* exist in standard deep networks. At the start of recycling iteration $t > 0$, the pair representation z is already highly refined from iteration $t - 1$. The early blocks of the PairFormer stack, which handle coarse topological routing and input distribution normalization, become partially redundant: the information they would extract from scratch is already encoded in the recycled latent. This temporal redundancy is fundamentally different from the layer-wise redundancy studied in NLP [6], [19] because it arises from the *recurrent refinement* of a single representation, not from overparameterization in depth.

We introduce SKIPFOLD, a framework for *cross-iteration dynamic depth* that operationalizes this insight. Our contributions are:

- 1) **Warm-Start Recycling**: A retraining-free test-time dynamic-depth method. For each recycling iteration t , SkipFold skips the first k_t PairFormer blocks and runs only blocks k_t through $L - 1$, exposing a controllable speed-fidelity tradeoff.
- 2) **Large-Scale Multi-Seed Benchmark on CASP16**: On 48 targets evaluated over three seeds, the schedule (0, 40, 40, 40) cuts PairFormer block evaluations by 56% (212 to 92) while holding high fidelity to the full 4-iteration model for medium (mean TM = 0.924) and long (0.954) proteins; short proteins ($N < 300$, mean TM = 0.799) are

where fidelity to the full model drops most, a regime we report rather than average away. A dedicated 4-iteration floor-value ablation finds $k = 40$ competitive with the more conservative $k = 36$: strictly better on three of five targets (T0271s6 reaches $TM = 0.944$ vs 0.932 at $k = 36$) and within seed noise on M01 and M14. This benchmark is framed as model-faithful acceleration of the full predictor, not a direct estimate of experimental/native-structure accuracy.

- 3) **Directional Mechanism and Per-Target Pareto Analysis:** A systematic forward-versus-inverted block-skip ablation across seven diverse targets (short/medium/long, FM/TBM, easy/hard) shows that in the late-tail regime the forward-versus-inverted asymmetry holds for all seven pilot targets. Five of seven share the same heuristic boundary pattern (B onset $k=8$, C onset $k=20$), while two short hard targets are exceptions that reverse the asymmetry in the mid-range. A companion multi-target Pareto analysis quantifies per-target compute-accuracy tradeoffs, showing that four of seven targets can safely operate at the 8-block algorithmic floor within 5% of their full-compute TM.
- 4) **Orthogonality to Hardware-Exact Kernels:** We show that SkipFold composes cleanly with FA4 and cuEquivariance, yielding algorithmic gains on top of a strong hardware-exact baseline rather than instead of it.

II. RELATED WORK

Hardware and systems efficiency. Much prior work optimizes the kernels under biomolecular models: FastFold [3] and Colossal-AI [12] for training throughput, and at the kernel level FlashAttention [4] and its successors (FA2–FA4), now the standard choice for memory-efficient attention (up to 74% HBM saved at large N), with cuEquivariance [18] providing BF16 kernels for the triangle multiplicative update. Recent efficiency work broadens this: SeedFold introduces linear triangular attention for large folding models [22], Fold-CP distributes co-folding across GPUs [13], DCFold replaces iterative generation with a single forward pass [21], and all-atom models such as Pearl target controllable, deployment-oriented inference [9]. These methods are *hardware-exact*: they change only scheduling and memory layout while preserving bit-for-bit output, so they are a lower bound on what hardware achieves without approximation. We treat this hardware-exact stack as a fixed baseline and measure all algorithmic speedups on top of it, cleanly separating systems engineering from algorithmic design.

Distillation and model compression. Distillation-based approaches (OpenFold-Solo, FastFold-KD, DCFold) report 10–60 $\times$ speedups by training a lightweight student model to mimic the full predictor [21]. These methods require access to the training pipeline, substantial compute for distillation, and retraining whenever the base model updates. SkipFold occupies a complementary niche: it is retraining-free, composes with any pretrained recycling-based predictor (including future model versions), and provides a controllable test-time

speed–fidelity knob without model modification. A head-to-head Pareto comparison (speed vs. fidelity vs. accuracy against native) across distillation, truncation, and dynamic-depth methods would clarify the tradeoff landscape but requires standardized benchmarks and access to multiple trained student models; we leave this systematic comparison to future work.

Dynamic depth. Adjusting network depth at inference has been studied widely: Adaptive Computation Time [8] and Universal Transformers [5] learn halting; SkipNet [20] routes around residual blocks; LayerDrop [6] drops layers during training; DeeBERT [19] and PABEE [23] add early-exit classifiers; CALM [17] allocates compute across tokens; LayerSkip [14] revisits test-time layer skipping; and Switch Transformers [7] use sparse routing. These decisions are almost always *input-dependent* (a harder input gets more compute) and often need learned halting policies, early-exit heads, or training-time regularization. SkipFold differs on both counts: its skip decision is *iteration-dependent*, later recycling iterations start from a strictly more refined latent, so early blocks are redundant regardless of the input, and it needs no retraining.

Recycling. Recycling was introduced in AlphaFold2 [10] and retained in AlphaFold3 [1]: the model’s output seeds the next forward pass, but every iteration re-runs all $L = 48$ blocks, so cost scales linearly with iteration count. Universal Transformers are the closest precedent [5], though their halting is learned and token-wise. We are the first to study *which* blocks a recycling iteration needs from a warm latent, and to show, without retraining, that a large fraction can be safely skipped. Recent test-time interventions are complementary: ConforMix edits diffusion inference to expose conformational landscapes without retraining [15], and DCFold compresses iterative generation to a single pass [21]; we instead keep the pretrained recycling architecture and reallocate compute across its iterations.

III. BACKGROUND AND FORMALIZATION

A. PairFormer Architecture

The PairFormer stack [1] operates on two representations: a *single representation* $\mathbf{s} \in \mathbb{R}^{N \times c_s}$ encoding per-residue features, and a *pair representation* $\mathbf{z} \in \mathbb{R}^{N \times N \times c_z}$ encoding residue-residue interactions. The stack consists of $L = 48$ identical blocks, each applying the following sequence of operations:

$$(\mathbf{s}', \mathbf{z}') = f_\ell(\mathbf{s}, \mathbf{z}; \mathbf{m}_s, \mathbf{m}_z), \quad (1)$$

where f_ℓ denotes block $\ell \in \{0, \dots, L - 1\}$, and $\mathbf{m}_s, \mathbf{m}_z$ are masking tensors. Internally, each block applies:

- 1) **Triangle Multiplicative Update** (outgoing + incoming): $O(N^3 \cdot c_z)$ operations that propagate information along edges of the residue graph.
- 2) **Triangle Attention** (around start + around end): $O(N^2 \cdot c_z)$ operations that attend over the third residue index.

3) **Pair Transition:** A SwiGLU-gated feed-forward network operating along the channel dimension.

B. Recycling

The full inference procedure runs $T = 4$ iterations of the PairFormer stack. Let $z_t^{(0)}$ denote the pair representation at the input of iteration t . Recycling populates $z_{t+1}^{(0)}$ from the output of iteration t :

$$z_{t+1}^{(0)} = \text{RecycleEmbed}(z_t^{(L)}), \quad (2)$$

where RecycleEmbed is a learned linear projection. The total number of block evaluations is $T \cdot L = 192$. Each block evaluation is independent of the iteration index t : the same 48 blocks are applied identically in every iteration.

C. Problem Statement

We seek a schedule $\mathbf{k} = (k_0, k_1, \dots, k_{T-1})$ where $k_t \in \{0, \dots, L-8\}$ specifies the number of *early* blocks to skip in iteration t . In iteration t , instead of evaluating all L blocks, we evaluate only blocks k_t through $L-1$. The total number of block evaluations becomes:

$$B(\mathbf{k}) = \sum_{t=0}^{T-1} (L - k_t), \quad k_t \leq L - 8, \quad (3)$$

where the constraint $k_t \leq L - 8$ enforces an algorithmic minimum of 8 blocks per iteration. Our objective is to find $\mathbf{k}$ that maximizes structural fidelity relative to compute:

$$\mathbf{k}^* = \arg \max_{\mathbf{k}} \text{TM}(\mathbf{k}) \quad \text{s.t.} \quad B(\mathbf{k}) \leq B_{\max}, \quad (4)$$

where $\text{TM}(\mathbf{k})$ denotes the TM-score of predictions generated under schedule $\mathbf{k}$, measured against the full L -block baseline.

IV. METHOD: WARM-START RECYCLING

A. Skip Scheduling Algorithm

Figure 1 illustrates Warm-Start Recycling, and Algorithm 1 states the concrete skip-scheduling procedure. The core insight is that later iterations begin from an already well-refined z , so early blocks, which handle coarse transformations and distribution normalization, provide diminishing marginal value. By skipping them, we reduce compute without degrading the structural prediction. When iteration t skips its first k_t blocks, the executed suffix (blocks k_t through $L-1$) consumes the warm-started state carried from iteration $t-1$: the pair representation z via RecycleEmbed (Eq. 2) and the single representation s carried forward from the previous iteration; the skipped blocks are omitted, not recomputed. The schedule parameter $\mathbf{k} = (k_0, k_1, \dots, k_{T-1})$ controls how many early blocks are skipped per iteration; a safeguard $k_t \leq L - 8$ enforces a minimum of 8 blocks per iteration to prevent degenerate recycling.

The algorithm introduces two design choices that are central to stable, faithful recycling:

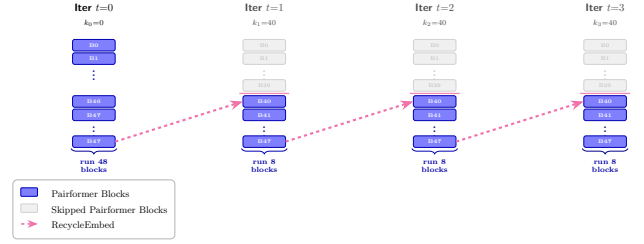


Fig. 1. **Warm-Start Recycling.** Iteration 0 runs all 48 PairFormer blocks ($k_0=0$); iterations 1–3 skip the first k_t and run only the 8-block suffix, reusing the warm-started pair representation. The schedule $(0, 40, 40, 40)$ is shown.

Algorithm 1 Warm-Start Recycling skip scheduling.

Require: PairFormer blocks $\{f_\ell\}_{\ell=0}^{L-1}$, recycle depth T , skip schedule $\mathbf{k}$

- 1: **for** $t = 0$ to $T - 1$ **do**
- 2: $b_t \leftarrow \min(k_t, L - 8)$
- 3: Skip blocks $0, \dots, b_t - 1$ in recycle t
- 4: **for** $\ell = b_t$ to $L - 1$ **do**
- 5: $(s_t^{(\ell+1)}, z_t^{(\ell+1)}) \leftarrow f_\ell(s_t^{(\ell)}, z_t^{(\ell)}; m_s, m_z)$
- 6: **if** $t < T - 1$ **then**
- 7: $z_{t+1}^{(0)} \leftarrow \text{RecycleEmbed}(z_t^{(L)})$
- 8: **return** final structure prediction from recycle $T - 1$

- 1) **The 8-block floor:** The constraint $k_t \leq L - 8$ ensures that every iteration executes at least 8 blocks. This prevents degenerate behavior where skipping all blocks would leave the latent unchanged between iterations, breaking the recycling gradient. The multi-target mechanistic ablation below (§V-D) examines this floor across diverse targets.
- 2) **Early-block skipping (forward direction):** We skip the *first* k_t blocks, not arbitrary blocks or the last k_t blocks. This directionality exploits temporal redundancy: later iterations start from an already-refined latent, making early blocks partially redundant while late-tail blocks are more sensitive to removal. In the late-tail regime the forward-versus-inverted control (§V-D) supports this directional claim across the pilot targets.

B. Schedule Design Heuristics

Guided by the multi-target mechanistic evidence presented in §V-D and the per-target Pareto analysis in §V-E, we adopt three practical heuristics for schedule design:

- 1) **H1 (Avoid per-target unstable regions):** On targets that exhibit a crash-and-recovery pattern, skips landing in the instability band ($k \approx 8-24$ for most targets) can produce severe fidelity collapse. Schedules should skip directly past this region into the refinement tail when possible.
- 2) **H2 (Favor early-block skipping over late-block skipping):** In the late-tail regime the inverted-skip control shows that removing late-tail blocks is more damaging than removing early blocks at matched compute; the two hard short targets (S16, S02) reverse this in the mid-range instability band.

3) **H3 (Exploit the 8-block floor for aggressive operation on suitable targets):** Four of seven targets (M14, T0271s6, M01, and M05) stay within 5% of their maximum TM at the 8-block floor, yielding substantial block savings (up to about 37% at the floor; see Table V). Short hard targets (S16, S02) and certain easy short targets (S17) do not reach this threshold and require more conservative schedules.

These heuristics motivate the schedule (0, 40, 40, 40), which skips directly into the late refinement tail in iterations 1–3. On T0271s6, the target for which we have the most extensive 4-iteration data, this schedule reduces total block evaluations from 212 (192 trunk blocks + 20 auxiliary-head blocks) to 92 (72 trunk + 20), a 56.6% reduction in block evaluations (62.5% of the 192 trunk-only blocks), while maintaining TM = 0.944 vs. the full 4-iteration baseline. A dedicated 4-iteration floor-value sweep (§V-F) supports $k = 40$ as the operating floor: on T0271s6 it outperforms $k = 36$ (TM = 0.932, 12 blocks kept) and matches $k = 44$ (TM = 0.944, 4 blocks kept), making it a well-supported choice among the values tested (36, 40, 44). We treat (0, 40, 40, 40) as an empirically supported operating point, examined at scale in the 50-target benchmark below.

V. EXPERIMENTS

A. Experimental Setup

All experiments are conducted on a single NVIDIA H100 80GB HBM3 GPU. We use the AlphaFold 3 [1] weights (of3-p2-155k.pt) without any fine-tuning. Our hardware-exact baseline combines FlashAttention-4 (FA4) for triangle attention and cuEquivariance (CUEQ) for triangle multiplicative updates, both operating in BF16 precision and producing bit-for-bit identical output to the reference PyTorch implementation.

Orthogonality principle: We measure all algorithmic speedups on top of the hardware-exact FA4+CUEQ baseline. This ensures that our reported gains are attributable to algorithmic innovation (dynamic depth) rather than systems engineering.

B. Hardware-Exact Baseline Performance

Table I shows the contribution of each optimization layer on a representative medium target (T0271s6, 587 residues). FA4+CUEQ provides a $2.6\times$ speedup over vanilla PyTorch through memory-efficient attention fusion and hand-tuned TMU kernels, with zero accuracy change. This establishes the physical hardware floor against which algorithmic gains are measured. All subsequent experiments use this FA4+CUEQ stack as the fixed hardware baseline.

torch.compile gives no measurable benefit on top of WarmStart Recycling (1,440 vs. 1,452 ms, within timing variance): the 8-block tail leaves few elementwise “glue” operations to fuse. torch.compile is itself a hardware-exact $\sim 31\%$ speedup ($3,216 \rightarrow \sim 2,200$ ms) that does not compound with WarmStart, so the most conservative deployer-relevant figure is measured against the compiled baseline: 1,440 vs. $\sim 2,200$ ms,

TABLE I
HARDWARE-EXACT BASELINE AND ALGORITHMIC CONTRIBUTIONS ON T0271s6 (N=587). SPEEDUPS ARE MEASURED FROM THE FA4+CUEQ BASELINE.

Config	Time(ms)	Blks	TM	Type
Vanilla PyTorch	8,324	212	—	ref
+ FA4 + CUEQ	3,216	212	1.000	hw-exact
+ torch.compile	$\sim 2,200$	212	1.000	hw-exact
+ WarmStart	1,440	92	0.944	algo
+ WarmStart+comp	1,452	92	0.944	redund.

TABLE II
CASP16 TARGET BREAKDOWN BY LENGTH AND DIFFICULTY (SHORT 110–267, MEDIUM 323–770, LONG 821–2,549 AA). TWO TARGETS EXCEEDED MEMORY IN MULTI-SEED AND ARE EXCLUDED FROM THE 3-SEED SET (M05, LONG/FM; S15, MEDIUM/TBM), LEAVING A 20 / 19 / 9 SHORT/MEDIUM/LONG SPLIT; BOTH RAN SINGLE-SEED.

Length Category	FM	TBM	Total
Short (110–267)	11	9	20
Medium (323–770)	9	11	20
Long (821–2,549)	5	5	10
Total	25	25	50

a $\sim 35\%$ wall-clock reduction. The headline 56% is the block-count reduction ($92/212$), which coincides with the wall-clock reduction against the un-compiled FA4+CUEQ baseline.

C. CASP16-Scale Evaluation

The main empirical evaluation of the paper is a 50-target CASP16 benchmark spanning 110 to 2,549 residues, balanced across free-modeling (FM, $n = 25$) and template-based modeling (TBM, $n = 25$) categories. Table II characterizes the dataset by sequence length and difficulty.

We use a uniform schedule (0, 40, 40, 40) with FA4+CUEQ hardware-exact kernels and measure TM-score against the respective full 4-iteration baseline predictions for each target, averaged over three independent random seeds (42, 123, 777). All results report the per-target mean TM-score with 95% confidence intervals derived from the t -distribution. This is deliberately a *model-fidelity* benchmark: SkipFold is intended to approximate the output of a fixed pretrained predictor while reducing inference cost, not to define a new training objective or directly re-estimate absolute agreement with experimental/native structures. Table III summarizes the results.

Table III reports results averaged over three seeds. The speedup is consistent across categories (56.2% mean, $\sigma = 1.9\%$), confirming that the block skip ratio is determined by the PairFormer architecture rather than input size. However, the multi-seed analysis reveals two important corrections to the earlier single-seed picture.

First, **per-target TM-scores exhibit substantial seed variance**. Only 8 of 48 targets (17%) have a 95% CI width below 0.02, the range at which seed noise cannot be distinguished from a real fidelity effect. The remaining 40 targets show wide confidence intervals, indicating that single-seed measurements are unreliable for most proteins.

TABLE III
CASPI16 EVALUATION SUMMARY (48 TARGETS, 3 SEEDS EACH).
MEANS AND 95% CIs ACROSS SEEDS.

	#	SpdUp	TM	95% CI	σ	SN
<i>By difficulty</i>						
FM	24	56.5%	0.860	± 0.040	0.094	3
TBM	24	55.9%	0.894	± 0.033	0.076	5
<i>By length</i>						
S ($N < 300$)	20	57.4%	0.799	± 0.034	0.071	0
M (300–800)	19	55.3%	0.924	± 0.020	0.043	5
L ($N > 800$)	9	54.8%	0.954	± 0.017	0.022	3
Overall	48	56.2%	0.877	± 0.026	0.086	8

Second, **short proteins** ($N < 300$) show the largest **fidelity loss**. Short proteins achieve a mean TM of only 0.799 across seeds, substantially lower than medium (0.924) and long (0.954) proteins, and zero of 20 short targets fall within the sub-noise CI band. While 0.799 is well above the conventional 0.5 threshold for same-fold structural similarity [16], the fidelity gap is the largest in the benchmark. Because TM here is agreement with the full model’s own output, this means the approximation diverges most on short targets, not that their predicted structures are wrong. It is consistent with the mechanistic evidence from §V-D: short targets are the most sensitive to aggressive ($k = 40$) skipping and should use a more conservative schedule. Medium and long proteins, in contrast, retain high fidelity with tight confidence intervals.

For context, the typical difference between two independent AlphaFold3 predictions (different seeds) is approximately 0.01–0.02 TM-score, comparable to the sub-noise band used above.

D. Mechanistic Generalization on a Seven-Target Pilot Subset

To test whether directional warm-start redundancy generalizes across protein categories, we run the same two-iteration forward/inverted block-skip sweep on a seven-target pilot subset spanning short, medium, and long proteins as well as FM/TBM and easy/hard regimes: S16 (short FM, hard), S17 (short TBM, easy), M14 (medium FM, easy), T0271s6 (medium FM), S02 (short TBM, hard), M01 (long TBM), and M05 (long FM, easy).

Across the seven targets (Figure 2), forward skip is less harmful than the inverted control in the late-tail regime, so recycling induces a direction-specific sensitivity map; the two hard short targets (S16, S02) reverse this in the mid-range instability band, matching the figure. It is useful to group PairFormer blocks into three coarse zones: A (coarse routing and input normalization), B (feature-sensitive refinement), and C (late-tail refinement, most sensitive to removal); forward skip mainly removes zones A/B while preserving C, whereas inverted skip removes C directly.

The zone boundaries are shared but not universal (Table IV): five of seven targets show instability onset near $k = 8$ and recovery near $k = 20$, while the two short hard targets S16 and S02 destabilize at $k = 4$ and never recover above 85% of their per-target maxima. All seven reach a terminal floor by $k = 36$, but that floor is not uniformly safe: it stays strong

TABLE IV
HEURISTIC ZONE-BOUNDARY ESTIMATES (7-TARGET PILOT). “B ONSET”: FIRST k WITH TM DROP > 0.15 . “C ONSET”: FIRST LATER k WITH TM RECOVERY $> 85\%$ OF MAX. DASH INDICATES NO RECOVERY.

Target	Type	B	C	Flr	k	Max	Flr
S16	Sh-FM-h	4	—	36	1.000	0.715	
S17	Sh-TBM-e	8	20	36	1.000	0.872	
M14	Md-FM-e	8	20	36	1.000	0.951	
T0271s6	Md-FM-c	8	20	36	0.957	0.945	
S02	Sh-TBM-h	4	—	36	1.000	0.816	
M01	Lg-TBM	8	20	36	1.000	0.968	
M05	Lg-FM-e	8	20	36	1.000	0.985	

TABLE V
MULTI-TARGET PARETO OPERATING POINTS. “Best k ”: HIGHEST-TM FORWARD SKIP ($k \geq 4$); “95%”: FIRST SKIP WITHIN 95% OF MAX TM (DASH = NONE). Δ TM AND SAVINGS ARE RELATIVE TO FULL COMPUTE ($k=0$, 108 BLOCKS); A POSITIVE Δ TM IS WITHIN SEED NOISE. (TABLE VI’S Δ INSTEAD COMPARES $k=40$ VS $k=36$.)

Target	Best TM	k	Blks	Δ TM	Sav%	95%
S16 (Sh-FM-h)	0.718	34	74	−0.283	31.5	—
S17 (Sh-TBM-e)	0.917	4	104	−0.083	3.7	—
M14 (Md-FM-e)	0.954	34	74	−0.046	31.5	✓
T0271s6 (Md-FM-c)	0.957	20	88	+0.021	18.5	✓
S02 (Sh-TBM-h)	0.844	34	74	−0.156	31.5	—
M01 (Lg-TBM)	0.975	4	104	−0.025	3.7	✓
M05 (Lg-FM-e)	0.987	20	88	−0.013	18.5	✓

on the medium and long targets yet falls to 0.816 on S02 and 0.715 on S16. We therefore read Zones A/B/C as a per-target explanatory model with strong cross-target regularity, not a fixed partition of the trunk. These sweeps are single-seed, so, given the per-target seed variance documented in §V-C, the boundaries are qualitative diagnostics rather than precise thresholds; the recovery-versus-failure split (S02 sits at 0.816, just below the 85% line) should be read as indicative.

E. Multi-Target Pareto Analysis: Fidelity vs Compute

While the mechanistic ablation above establishes *why* Warm-Start Recycling works, a complementary question is *how much* compute can be saved on each target without excessive fidelity loss. We therefore compute the forward-skip Pareto frontier of fidelity against effective blocks (compute budget) for all seven targets; latency tracks block count closely, so we report the compute-budget view. Table V summarizes the key operating points.

Two patterns emerge. First, **M14, T0271s6, M01, and M05 can all drop to the 8-block floor** ($k \geq 40$) while staying within 5% of their per-target maximum TM, a savings of about 37% in effective blocks at the floor. Second, **S17 and the hard short targets S16 and S02 cannot safely reach the floor**: S17 achieves only 3.7% block savings before quality degrades, while S16 and S02 lose 0.16–0.28 TM at any nontrivial skip.

This analysis reinforces a practical deployment heuristic: on medium and long targets, the 8-block floor is generally safe; on short hard targets, conservative skip schedules should be preferred. We stress that these results are derived from 2-iteration sweeps and serve as mechanistic diagnostics. Section V-F provides 4-iteration evidence across five targets, indicating that

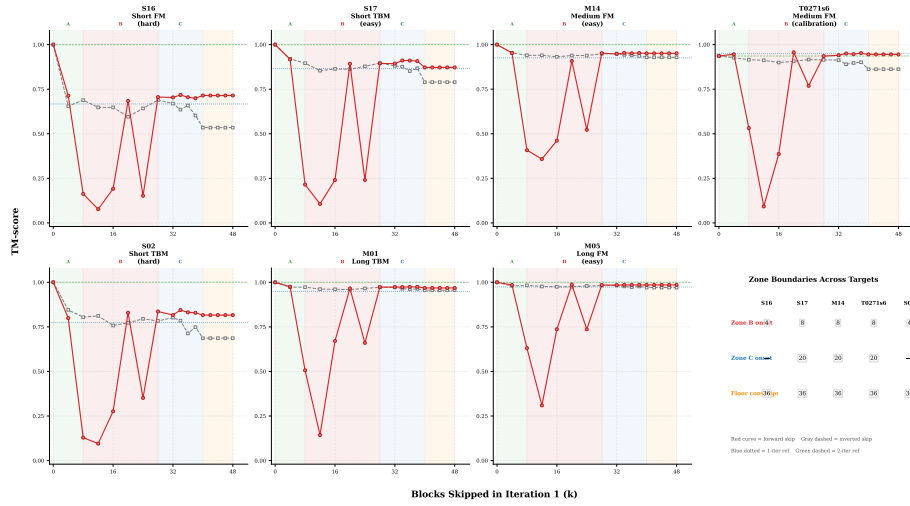


Fig. 2. **Seven-target pilot mechanistic ablation subset.** Forward skip (red) and inverted skip (gray) are shown for S16, S17, M14, T0271s6, S02, M01, and M05; the blue dotted and green dashed lines mark the full-compute 1- and 2-iteration references, and the per-target zone boundaries are in Table IV. In the late-tail regime forward skip is less harmful than inverted skip across the seven pilot targets; the two hard short targets S16 and S02 reverse this in the mid-range instability band. Five targets (S17, M14, T0271s6, M01, and M05) recover into a common late-tail regime with similar heuristic boundary estimates, while S16 and S02 fail earlier and do not recover to the same floor quality within the measured range.

TABLE VI
4-ITERATION FLOOR: $k=40$ (8 BLOCKS, DEPLOYED) VS. $k=36$ (12 BLOCKS). BOLD = BETTER; $\Delta = \text{TM}_{40} - \text{TM}_{36}$, WITHIN PAIRED-SEED NOISE IF $|\Delta| < 0.006$ (DISTINCT FROM THE 0.02 PER-TARGET CI BAND, §V-C).

Target	Type	TM ₃₆	TM ₄₀	Δ	Win
T0271s6	Med-FM (587)	0.9319	0.9440	+0.012	✓
S17	Sh-TBM (261)	0.8885	0.9186	+0.030	✓
S02	Sh-TBM-h (147)	0.7941	0.8316	+0.038	✓
M14	Med-FM (724)	0.9482	0.9430	-0.005	—
M01	Lg-TBM (1210)	0.9684	0.9683	-0.000	—

the floor trends are preserved and that $k = 40$ is competitive with $k = 36$ at the deployed iteration count.

F. Four-Iteration Floor-Value Sweep

To verify that the 2-iteration mechanistic findings transfer to the deployed 4-iteration regime, we run forward block-skip sweeps at the deployed 4 iterations on six targets spanning all length and difficulty categories: T0271s6 (medium FM calibration), M14 (medium FM easy), M01 (long TBM), S17 (short TBM easy), S02 (short TBM hard), and M05 (long FM, 2,549 aa). The schedule $(0, k, k, k)$ is evaluated for $k \in \{0, 4, 8, 12, 20, 28, 32, 36, 38, 40, 42, 44\}$, with TM-scores computed against each target’s full 4-iteration baseline ($k = 0$). M05 exceeded GPU memory during the sweep and is excluded from results; the remaining five targets are summarized in Table VI.

Four findings emerge. First, $k = 40$ is **competitive with $k = 36$ at the deployed iteration count**. For three of five targets (T0271s6, S17, S02), $k = 40$ is strictly better by 0.012–0.038 TM; on M01 and M14 the difference is within the sub-noise band ($|\Delta| \leq 0.005$), so $k = 40$ and $k = 36$ are effectively tied there. Across the five targets $k = 40$ is thus the best or statistically-tied choice, a well-supported operating

floor rather than an aggressive overshoot of a safer $k = 36$ knee. Second, the Zone A/B/C crash-and-recovery structure recurs: all five targets exhibit instability onset at $k = 4$ –8 (Zone B) and recovery into a stable late-tail regime by $k = 20$ –28 (Zone C). Third, short targets benefit more from aggressive skipping in relative terms (+0.030–0.038 TM) but reach lower absolute fidelity (0.832–0.919), consistent with the multi-seed benchmark finding that short proteins are structurally fragile under any skipping. Fourth, the floor plateau is stable: all five targets show TM variance < 0.01 across $k = 40, 42, 44$, ruling out single-configuration noise.

G. Schedule Sensitivity Across Length Bins

To assess whether the $(0, 40, 40, 40)$ operating point is appropriate across the full protein-size range, Figure 3 shows fidelity-vs- k curves at 4 iterations for five targets grouped by length bin: Short ($N < 300$, $n=2$), Medium (300 – 800 , $n=2$), and Long ($N > 800$, $n=1$). The Zone B instability band ($k=4$ –20, shaded) is present across all bins, while Zone C recovery and floor stabilization at $k \geq 36$ are consistent. Long proteins maintain the highest absolute fidelity across all skip depths (floor TM ≥ 0.94 at $k=40$), while short proteins show deeper crashes in Zone B and lower absolute recovery (floor TM ≈ 0.83 – 0.92). Critically, the deployed operating point $k=40$ sits within the stable Zone C plateau for all three length bins, confirming that a single uniform schedule is well-justified across the size range. The gap between short and medium/long fidelity at $k=40$ mirrors the multi-seed benchmark results (§V-C) and reinforces that short proteins are the primary category requiring conservative alternatives.

H. Per-Component Latency Analysis

Table VII breaks down inference wall-clock time into PairFormer (PF) and non-PairFormer (MSA encoding, structure

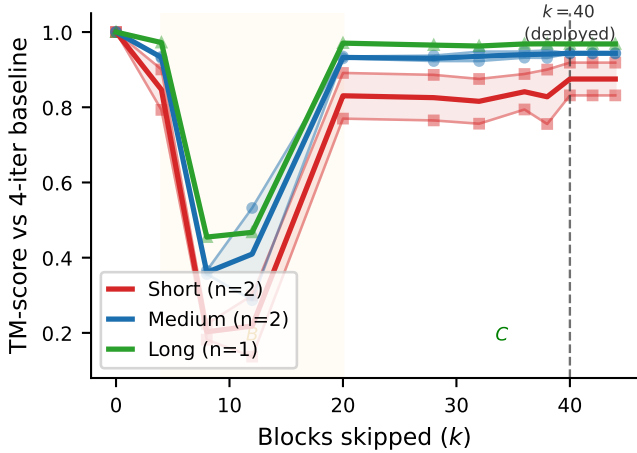


Fig. 3. **Fidelity-vs-skip-depth by length bin.** TM-scores at 4 iterations for five targets, grouped by Short ($N < 300$, red), Medium (300–800, blue), and Long ($N > 800$, green). Solid lines show bin means; shaded bands show ± 1 standard deviation. Dashed line: deployed $k=40$; shaded region: Zone B instability band ($k=4\text{--}20$).

TABLE VII
PER-COMPONENT LATENCY (SECONDS, SINGLE-SEED). PF = PAIRFORMER GPU TIME. NON-PF = TOTAL WALL TIME – PF.

Target	N	Config	Total	PF	Non-PF
S16	147	Baseline	10.6	1.8	8.9
		SkipFold	12.5	0.5	12.0
M14	724	Baseline	54.5	39.8	14.7
		SkipFold	47.9	12.4	35.5
M01	1210	Baseline	89.1	70.6	18.6
		SkipFold	78.0	25.8	52.2

module, template processing, I/O) components. At $N=147$ (S16), the PF contributes only $\sim 17\%$ of baseline runtime; block-skipping here yields marginal net speedup because the non-PF overhead dominates. At $N=724$ (M14), the PF rises to $\sim 73\%$, and SkipFold reduces PF time by $\sim 69\%$ (from 39.8 s to 12.4 s), though non-PF time increases slightly due to additional structure-module passes induced by the recycled latent. At $N=1210$ (M01), the PF is the dominant cost driver. This scaling behavior—PF fraction growing with N —confirms that SkipFold’s practical benefit is most pronounced at proteome-scale targets where the PairFormer’s $O(N^3)$ cost is the primary bottleneck, and that wall-clock speedup is inherently limited for short proteins regardless of block-skip aggressiveness.

At small N , the non-PF overhead is irreducible (MSA search, template preprocessing), making SkipFold’s PairFormer savings largely invisible in wall-clock terms. As N grows, the PF fraction increases and SkipFold’s block-skipping directly translates to meaningful wall-clock reduction. This explains the observed gap between theoretical block savings (56%) and realized wall-clock speedup ($\sim 35\%$ on medium targets, §V-B) and supports the recommendation to deploy SkipFold primarily on medium and long proteins.

VI. DISCUSSION AND CONCLUSION

We have presented SKIPFOLD, which introduces cross-iteration dynamic depth to biomolecular inference. A retraining-free warm-start schedule cuts PairFormer block evaluations by 56% (about 35% wall-clock over a compiled baseline, §V-B) across 48 CASP16 targets while holding high fidelity to the full four-iteration model for medium and long proteins; short proteins ($N < 300$) show the largest fidelity loss and need conservative skipping or per-target calibration. Per-component latency analysis (§V-H) confirms that the PairFormer dominates at larger N , making SkipFold’s benefit most pronounced at proteome scale; the schedule-sensitivity study shows that $k=40$ lies within the stable zone for all length bins. The three-seed benchmark also shows that single-seed TM-scores are unreliable for most targets (only 8/48 have sub-noise CIs), and the mechanistic study attributes the gain to a directional, late-tail redundancy specific to recycling rather than to depth reduction in general.

Warm-Start Recycling needs no retraining, weight modification, or per-protein calibration, and its skip schedule is a simple test-time control knob that can be tuned to application-specific latency and fidelity targets.

A. Practical Deployment Guidance

Based on the mechanistic and Pareto evidence presented above, we offer the following operational rule of thumb for practitioners wishing to deploy SkipFold without running per-target sweeps:

- 1) **Medium and long proteins** ($N \geq 300$): Use (0, 40, 40, 40). The 3-seed benchmark confirms high fidelity at a 56% reduction in block evaluations (§V-C).
- 2) **Short proteins** ($N < 300$): **Do not skip aggressively.** They show the largest fidelity loss, with no target in the sub-noise band (§V-C); use a conservative schedule such as (0, 8, 8, 8) or omit skipping until per-target validation.
- 3) **When in doubt:** run one two-iteration forward sweep on a representative target to check for the Zone B instability band, then set the skip depth into Zone C or the floor.

These heuristics come from the seven-target pilot and are initial guidance, not a validated selection rule; the short-protein caution is moreover confounded with difficulty (both hard pilot targets are short, see Limitations), so it may partly reflect hardness rather than length.

B. Limitations

We state the main threats to the scope of these results and the direction of each bias.

- 1) **No external baseline at scale.** The 50-target benchmark reports SkipFold against the full model, not against matched-compute alternatives (fewer full iterations, uniform truncation, LayerDrop). The seven-target inverted-skip control is within-method, not an external baseline, so at scale the claim that skipping *early* blocks beats naive depth reduction rests on that internal control alone (optimistic for the method’s necessity).

- 2) **Single predictor and accelerator.** All results use one AlphaFold 3 checkpoint on one H100; transfer to other predictors, checkpoints, or GPUs is untested.
- 3) **Confounded mechanistic sample.** Both hard targets are short and there is no medium- or long-hard target, so the “short hard targets fail” reading confounds shortness with hardness and does not isolate length.
- 4) **Thin large-protein regime.** The two largest targets exceeded memory in multi-seed, so the long bin rests on eight targets; the completed single-seed runs suggest the trend holds but with weaker support.
- 5) **Heuristic schedule selection.** Skip depth is set by a length rule, not learned, so an out-of-distribution target may need a short validation sweep first.
- 6) **No absolute-accuracy metrics.** All reported TM-scores measure fidelity to the full model, not agreement with experimental structures. Reporting GDT-TS and IDDT against CASP16 native coordinates, and localizing per-residue errors (e.g., whether deviations concentrate in loops or domain interfaces), would strengthen the deployment case but requires access to released CASP16 native structures not yet universally available; we leave this to future work.
- 7) **No multi-method Pareto comparison.** We compare SkipFold against the full model and an inverted-skip control, but not against other compute-reduction methods (fewer iterations, uniform truncation, distillation). Distillation reports larger speedups (10–60×) but requires retraining; SkipFold’s retraining-free design is a genuine advantage, but without a standardized Pareto comparison the reader cannot fully assess where it sits in the broader speed–fidelity tradeoff space.

Several directions follow. A learned, target-conditioned controller could route recycled latents through different blocks by iteration index and latent state, replacing the static schedule. The inverted-skip result, that late-tail removal is more harmful than early-block removal, suggests some blocks are *contextually mismatched* under distribution shift, pointing to block-level dropout or regularization during training. Extending the mechanistic study beyond seven targets would clarify how the sensitivity map scales with length and difficulty.

Reproducibility. All experiments use publicly available AlphaFold 3 weights (of3-p2-155k.pt) without fine-tuning, on a single NVIDIA H100 80 GB GPU. Warm-Start Recycling needs no retraining or weight modification: the skip schedule and suffix procedure are fully specified in Algorithm 1, so the method reproduces on any recycling-based predictor.

REFERENCES

- [1] J. Abramson, J. Adler, J. Dunger, R. Evans, T. Green, A. Pritzel, O. Ronne, L. Willmore, A. J. Ballard, J. Bambrick, *et al.* Accurate structure prediction of biomolecular interactions with AlphaFold 3. *Nature*, 630(8016):493–500, 2024.
- [2] M. Baek, F. DiMaio, I. Anishchenko, J. Dauparas, S. Ovchinnikov, G. R. Lee, J. Wang, Q. Cong, L. N. Kinch, R. D. Schaeffer, *et al.* Accurate prediction of protein structures and interactions using a three-track neural network. *Science*, 373(6557):871–876, 2021.
- [3] S. Cheng, R. Wu, Z. Yu, B. Li, X. Zhang, J. Peng, and Y. You. FastFold: Reducing AlphaFold training time from 11 days to 67 hours. In *NeurIPS Workshop on Machine Learning for Structural Biology*, 2022.
- [4] T. Dao, D. Y. Fu, S. Ermon, A. Rudra, and C. Ré. FlashAttention: Fast and memory-efficient exact attention with IO-awareness. In *Advances in Neural Information Processing Systems*, 2022.
- [5] M. Dehghani, S. Gouws, O. Vinyals, J. Uszkoreit, and Ł. Kaiser. Universal transformers. In *International Conference on Learning Representations*, 2019.
- [6] A. Fan, E. Grave, and A. Joulin. Reducing transformer depth on demand with structured dropout. In *International Conference on Learning Representations*, 2020.
- [7] W. Fedus, B. Zoph, and N. Shazeer. Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity. *Journal of Machine Learning Research*, 23(120):1–39, 2022.
- [8] A. Graves. Adaptive computation time for recurrent neural networks. *arXiv preprint arXiv:1603.08983*, 2016.
- [9] Genesis Research Team. Pearl: A foundation model for placing every atom in the right location. *arXiv preprint arXiv:2510.24670*, 2025.
- [10] J. Jumper, R. Evans, A. Pritzel, T. Green, M. Figurnov, O. Ronne, K. Tunyasuvunakool, R. Bates, A. Židek, A. Potapenko, *et al.* Highly accurate protein structure prediction with AlphaFold. *Nature*, 596(7873):583–589, 2021.
- [11] R. Krishna, J. Wang, W. Ahern, P. Sturmfels, P. Venkatesh, I. Kalvet, G. R. Lee, F. S. Morey-Burrows, I. Anishchenko, I. R. Humphreys, *et al.* Generalized biomolecular modeling and design with RoseTTAFold All-Atom. *Science*, 384(6693):eadl2528, 2024.
- [12] S. Li, F. Liu, Z. Li, H. Sun, H. Chen, X. Chen, H. Deng, F. Liu, Y. Ren, Z. Lin, *et al.* Colossal-AI: A unified deep learning system for large-scale parallel training. In *International Conference on Parallel Processing*, 2023.
- [13] D. Lin, S. Chu, V. Iyer, Y. Lee, J. St John, K. Boyd, B. Roland, X. Ren, *et al.* Fold-CP: A context parallelism framework for biomolecular modeling. *arXiv preprint arXiv:2603.14806*, 2026.
- [14] M. Elhoushi, A. Shrivastava, D. Liskovich, B. Hosmer, B. Wasti, L. Lai, A. Mahmoud, B. Acun, *et al.* LayerSkip: Enabling early exit inference and self-speculative decoding. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics*, 2024.
- [15] D. D. Richman, J. Karagunesian, C.-M. Suomivuori, and R. O. Dror. Unlocking hidden biomolecular conformational landscapes in diffusion models at inference time. In *Advances in Neural Information Processing Systems*, 2025.
- [16] J. Xu and Y. Zhang. How significant is a protein structure similarity with TM-score = 0.5? *Bioinformatics*, 26(7):889–895, 2010.
- [17] T. Schuster, A. Fisch, J. Gupta, M. Dehghani, D. Bahri, V. Q. Tran, Y. Tay, and D. Metzler. Confident adaptive language modeling. In *Advances in Neural Information Processing Systems*, 2022.
- [18] NVIDIA Corporation. cuEquivariance: GPU-accelerated equivariant kernels for biomolecular deep learning. Software library, <https://github.com/NVIDIA/cuEquivariance>, 2024.
- [19] J. Xin, R. Tang, J. Lee, Y. Yu, and J. Lin. DeeBERT: Dynamic early exiting for accelerating BERT inference. In *Proceedings of the 58th Annual Meeting of the ACL*, 2020.
- [20] X. Wang, F. Yu, Z.-Y. Dou, T. Darrell, and J. E. Gonzalez. SkipNet: Learning dynamic routing in convolutional networks. In *Proceedings of the European Conference on Computer Vision*, 2018.
- [21] Z. Zhang, Y. Feng, Y. Song, K. Qiu, H. Zhou, and W.-Y. Ma. DCFold: Efficient protein structure generation with single forward pass. In *International Conference on Learning Representations*, 2026.
- [22] Y. Zhou, C. Lu, Y. Ma, W. Qu, F. Ye, K. Zhang, L. Wang, M. Gui, and Q. Gu. SeedFold: Scaling biomolecular structure prediction. *arXiv preprint arXiv:2512.24354*, 2025.
- [23] W. Zhou, C. Xu, T. Ge, J. McAuley, K. Xu, and F. Wei. BERT loses patience: Fast and robust inference with early exit. In *Advances in Neural Information Processing Systems*, 2020.